

Programming Assignment 1: Matching and Verifying

1/30/2026

100 Points Possible



Attempt 1



In Progress

NEXT UP: Submit Assignment



Add Comment

Unlimited Attempts Allowed

1/15/2026 to 2/2/2026

▼ Details

Due date: Jan 30, 2026 at 11:59PM.

In this assignment, you will implement the Gale Shapley algorithm for the hospital-student stable matching problem, and a separate verifier that checks whether a proposed matching is valid and stable. Your implementation must handle edge cases, such as empty files and one hospital and one student, and also check that the input is valid (i.e., equal number of hospitals and students).

Collaboration policy: Work in teams of 2 unless otherwise stated by the instructor. Both partners must contribute meaningfully and understand the full solution. Both partners need to do three meaningful github commits to the codebase. You can able to use any programming language, and standard libraries for parsing and I/O.

Task A: Matching Engine: Implement the hospital-proposing deferred acceptance algorithm

- Initially, all hospitals are unmatched and have not proposed to anyone.
- While there exists an unmatched hospital that still has students left to propose to:
 - The hospital proposes to the next student on its preference list that it has not yet proposed to.
 - The student tentatively accepts the best hospital (according to the student's preferences) among its current tentative match (if any) and the new proposer, rejecting the other.

Your program must output the final matching and (optionally) the number of proposals made. See the pseudocode given in class for specific detail.

Task B: Verifier

Write a separate program (or a separate mode in the same program) that:

(a) Checks validity: each hospital and each student is matched to exactly one partner, with no duplicates. And (b) checks stability: confirms there is no blocking pair.

Task C: Scalability

Measure the running time of your matching engine on an increasingly larger number of hospitals/students, i.e., $n = 1, 2, 4, 8, 16, 32, 64, 128, 256, 512$ and graph the running time as a line graph when n on the x-axis and the running time on the y-axis. Do the same for the verifier. What is the trend that you notice? Note: How you measure the running time is up to you (there are multiple ways of doing this) and will likely depend on which programming language you choose.

Input/Output Specification

Input Format: The input describes preferences for a one-to-one market with complete strict rankings.

- First line: integer n .
- Next n lines: hospital preference lists.
- Next n lines: student preference lists.

Hence, each preference line contains n integers, a permutation of $1..n$.

Output Format for Matching Algorithm

Output n lines, one per hospital i :

$i\ j$

meaning hospital i is matched to student j .

Output Format for Verifier Algorithm

Your verifier should print either:

- VALID STABLE or
- a clear failure message, e.g. INVALID (with reason) or UNSTABLE (with an example blocking pair). If it is both UNSTABLE and INVALID then you can output both or one, your choice.

Example Input: **example.in**

```
3
1 2 3
2 3 1
2 1 3
2 1 3
1 2 3
1 2 3
```

One Valid Output: **example.out**

```
1 2
2 3
3 1
```

Submission and Deliverables (GitHub):

As discussed in class, you will submit your work as a GitHub repository on Canvas. Only one of the partners should submit. Your repository must include:

1. Source code for:
 1. The matcher (hospital-proposing Gale--Shapley), and
 2. The `\textbf{verifier}` (validity + stability checker).
1. Example inputs and outputs: At least one example input file and the corresponding expected output file.
2. Your README must point to these files, explain how to reproduce the output and have the following:
 - Both students' names and UFIDs.
 - Instructions to compile/build your code (if applicable).
 - Instructions to run the matcher and the verifier, including example commands.
 - Any assumptions (input/output format, dependencies, etc.).
 - Your graph and solution to Task C.

Tips:

- Make sure you keep your repository structure clean and organized. i.e., Use a clean layout (e.g., `src/`, `data/`, `tests/`). Meaningful filenames; do not submit an unstructured dump of files
- A grader must be able to git clone the repository and follow the README to compile (if needed) and run your programs without additional steps.

Choose a submission type



Submit Assignment